SQL Subquery — Answer Key

Topic 1: What is a Subquery / Basics

1.

> Inner query: `(SELECT MAX(score) FROM movies)` — runs first, returns a single value (the highest score).

> Outer query: `SELECT * FROM movies WHERE score = ...` — uses that value to filter rows.

2.

```
SELECT * FROM movies
WHERE rating = (SELECT MAX(rating) FROM movies);
```

Topic 2: Independent Subquery — Scalar

3.

```
-- Subquery approach
SELECT * FROM movies
WHERE (gross - budget) = (SELECT MAX(gross - budget) FROM movies);

-- ORDER BY approach (returns only 1 row even if there's a tie)
SELECT * FROM movies
ORDER BY (gross - budget) DESC
LIMIT 1;
```

4.

```
SELECT COUNT(*) AS above_average_count
FROM movies
WHERE rating > (SELECT AVG(rating) FROM movies);
```

5.

```
SELECT * FROM movies
WHERE release_year = 2000
AND score = (SELECT MAX(score) FROM movies WHERE release_year = 2000);
```

6.

```
SELECT * FROM movies
WHERE score = (
    SELECT MAX(score) FROM movies
    WHERE votes > (SELECT AVG(votes) FROM movies)
);
```

7.

```
SELECT model, rating,
    ABS(rating - (SELECT AVG(rating) FROM smartphones WHERE brand = 'Samsung')) AS diff_from_avg
FROM smartphones
```

```
WHERE brand = 'Samsung';
```

Topic 3: Independent Subquery — Row

8.

```
SELECT * FROM users
WHERE user_id NOT IN (
    SELECT DISTINCT user_id FROM orders
);
```

9.

```
SELECT * FROM movies
WHERE director IN (
    SELECT director FROM movies
    GROUP BY director
    ORDER BY SUM(gross) DESC
    LIMIT 3
);
```

10.

```
SELECT * FROM movies
WHERE star IN (
    SELECT star FROM movies
    WHERE votes > 25000
    GROUP BY star
    HAVING AVG(score) > 8.5
);
```

Topic 4: Independent Subquery — Table

11.

```
SELECT * FROM movies
WHERE (release_year, gross - budget) IN (
    SELECT release_year, MAX(gross - budget)
    FROM movies
    GROUP BY release_year
);
```

12.

```
SELECT * FROM movies
WHERE (genre, score) IN (
    SELECT genre, MAX(score)
    FROM movies
    WHERE votes > 25000
    GROUP BY genre
);
```

13.

```
SELECT * FROM movies
```

```
WHERE (star, director) IN (
    SELECT star, director
    FROM movies
    GROUP BY star, director
    ORDER BY SUM(gross) DESC
    LIMIT 5
);
```

Topic 5: Correlated Subquery

14.

```
SELECT * FROM movies m1
WHERE score > (
    SELECT AVG(score) FROM movies m2
    WHERE m2.genre = m1.genre
);
```

15.

```
WITH fav_food AS (
    SELECT t2.user_id, name, f_name, COUNT(*) AS frequency
    FROM users t1
    JOIN orders t2 ON t1.user_id = t2.user_id
    JOIN order_details t3 ON t2.order_id = t3.order_id
    JOIN food t4 ON t3.f_id = t4.f_id
    GROUP BY t2.user_id, t3.f_id
)
SELECT * FROM fav_food f1
WHERE frequency = (
    SELECT MAX(frequency) FROM fav_food f2
    WHERE f2.user_id = f1.user_id
);
```

Topic 6: Usage with SELECT

16.

```
SELECT name, (votes / (SELECT SUM(votes) FROM movies)) * 100 AS vote_percentage
FROM movies;
```

17.

```
SELECT name, genre, score,
    (SELECT AVG(score) FROM movies m2 WHERE m2.genre = m1.genre) AS avg_genre_score
FROM movies m1;
```

> **Why inefficient:** This is a correlated subquery in the SELECT clause — it re-executes once for **every row** in the outer query. On 1000 movies, that's 1000 separate subquery runs. A `JOIN` with a pre-aggregated `GROUP BY` subquery (Table Subquery / FROM usage) computes the genre averages **once**, then joins — far fewer total operations.

Topic 7: Usage with FROM

18.

```
SELECT r_name, avg_rating
FROM (
    SELECT r_id, AVG(restaurant_rating) AS avg_rating
    FROM orders
    GROUP BY r_id
) t1
JOIN restaurants t2 ON t1.r_id = t2.r_id;
```

19.

```
SELECT t2.user_id, name, f_name, COUNT(*) AS frequency
FROM users t1
JOIN orders t2 ON t1.user_id = t2.user_id
JOIN order_details t3 ON t2.order_id = t3.order_id
JOIN food t4 ON t3.f_id = t4.f_id
GROUP BY t2.user_id, t3.f_id;

-- Then wrap as subquery in FROM and filter for max per user
SELECT * FROM (
    SELECT t2.user_id, name, f_name, COUNT(*) AS frequency
    FROM users t1
    JOIN orders t2 ON t1.user_id = t2.user_id
    JOIN order_details t3 ON t2.order_id = t3.order_id
    JOIN food t4 ON t3.f_id = t4.f_id
    GROUP BY t2.user_id, t3.f_id
) fav_food f1
WHERE frequency = (SELECT MAX(frequency) FROM fav_food f2 WHERE f2.user_id = f1.user_id);
```

Topic 8: Usage with HAVING

20.

```
SELECT genre, AVG(score)
FROM movies
GROUP BY genre
HAVING AVG(score) > (SELECT AVG(score) FROM movies);
```

21.

```
SELECT brand, AVG(rating)
FROM smartphones
GROUP BY brand
HAVING COUNT(*) > 20;
```

> Note: this particular question doesn't need a subquery — `COUNT(*) > 20` is a fixed number, not something computed from another query. A subquery would be needed only if the threshold itself was dynamic, e.g., `HAVING COUNT(*) > (SELECT AVG(phone_count) FROM ...)`.

Topic 9: Subquery in INSERT

22.

```
INSERT INTO loyal_users (user_id, name)
SELECT t1.user_id, t1.name
FROM orders t1
JOIN users t2 ON t1.user_id = t2.user_id
GROUP BY user_id
HAVING COUNT(*) > 3;
```

Topic 10: Subquery in UPDATE

23.

```
UPDATE loyal_users
SET money = (
    SELECT SUM(amount) * 0.1
    FROM orders
    WHERE orders.user_id = loyal_users.user_id
);
```

> This is a **correlated subquery** — the inner query references `loyal_users.user_id`, so it recalculates separately for each row being updated.

Topic 11: Subquery in DELETE

24.

```
DELETE FROM users
WHERE user_id NOT IN (
    SELECT DISTINCT user_id FROM orders
);
```

Topic 12: Mixed / Challenge

25.

```
SELECT brand, AVG(ram) AS avg_ram
FROM smartphones
WHERE refresh_rate >= 90 AND has_fast_charging = TRUE
GROUP BY brand
HAVING COUNT(*) >= 10
ORDER BY avg_ram DESC
LIMIT 3;
```

> No subquery needed — all conditions (`refresh_rate`, `fast_charging`, `count >= 10`) are **fixed thresholds**, not values derived from another query. Subqueries become necessary only when the comparison value itself must be calculated (like an average or max from the data).

26.

```
SELECT brand, AVG(price) AS avg_price
FROM smartphones
WHERE has_5g = TRUE
GROUP BY brand
HAVING AVG(rating) > 70 AND COUNT(*) > 10;
```

27.

```
SELECT * FROM movies
WHERE votes > (SELECT AVG(votes) FROM movies)
AND score = (
    SELECT MAX(score) FROM movies
    WHERE votes > (SELECT AVG(votes) FROM movies)
);
```

> Combines two scalar subqueries: one for the votes threshold, one for the max score within that filtered set.

28.

```
-- Correlated version (recalculates restaurant-specific avg per row comparison)
SELECT r_id, AVG(restaurant_rating) AS avg_rating
FROM orders o1
GROUP BY r_id
HAVING AVG(restaurant_rating) > (
    SELECT AVG(restaurant_rating) FROM orders o2
);
```

> This uses an **independent** subquery for the platform-wide average since it doesn't reference the outer query's current row — it's calculated once and reused for every group comparison in `HAVING`. A correlated version isn't needed here because the comparison value (platform-wide average) doesn't change per restaurant.